CYBERSECURITY
PRESENTATION

VULNERABLENOTES:

BUILDING, BREAKING & SECURING A FLASK APP

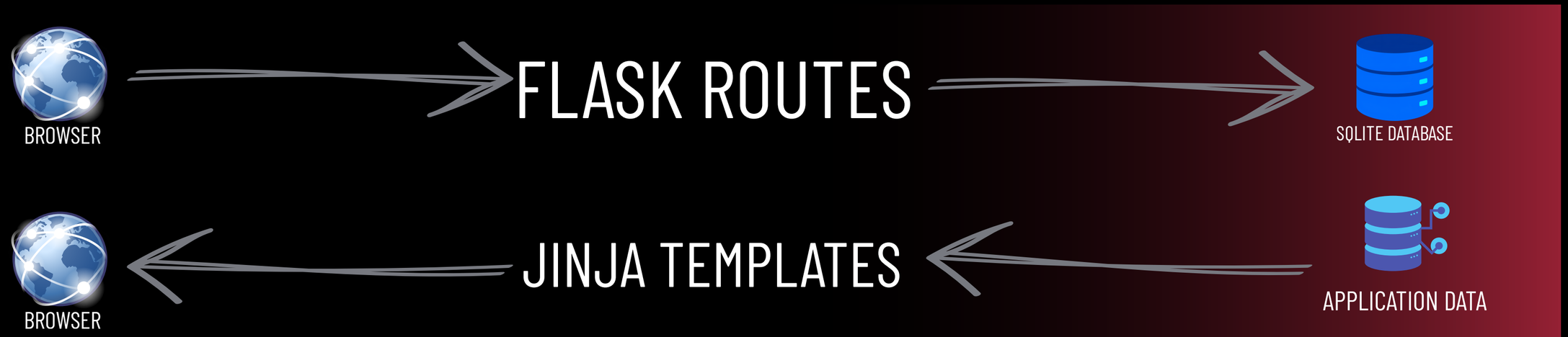
COMP6841 SECURITY ENGINEERING PROJECT

Flask · SQLite · Python

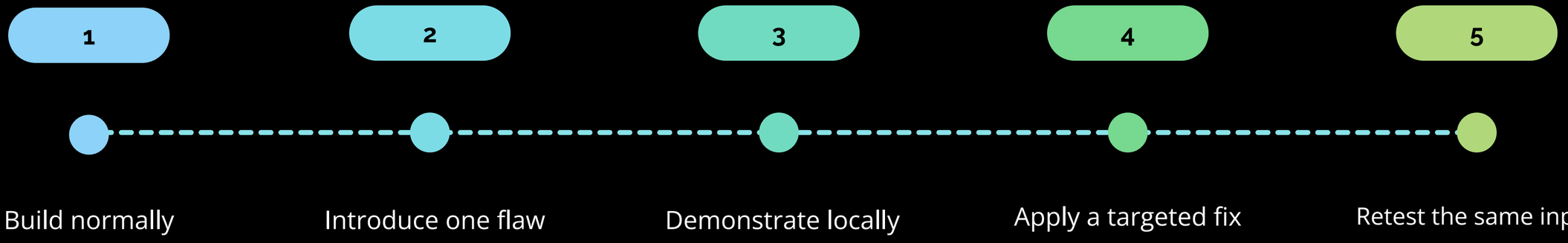
SQL Injection · Stored XSS · IDOR · Plaintext Passwords — Exploited & Fixed

By Zack Zhou

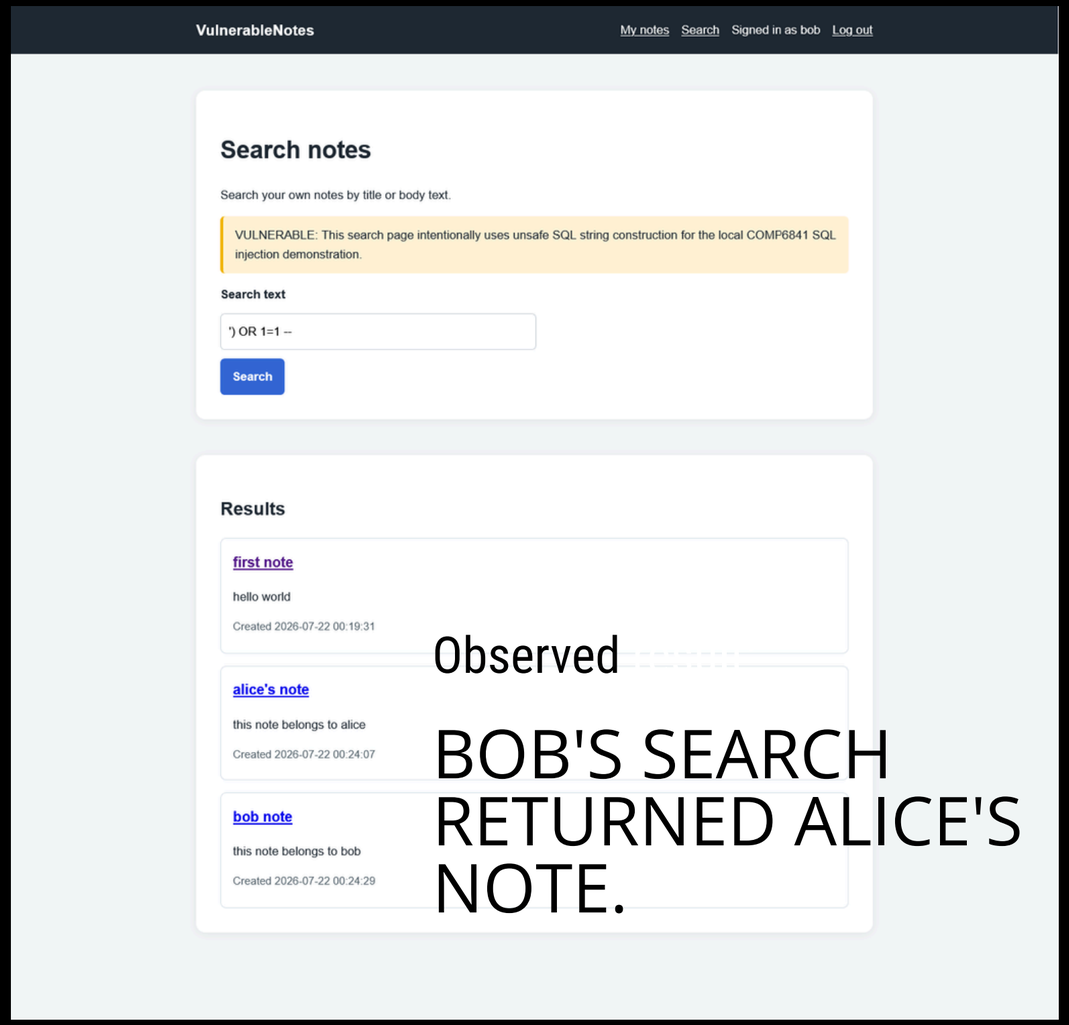
ONE APP, TWO VERSIONS, FOUR SECURITY TESTS



FEATURES: ACCOUNTS, SESSIONS, NOTE CREATION, NOTE VIEWING, AND SEARCH



SQL INJECTION EXPOSED OTHER USERS' NOTES



Observed

BOB'S SEARCH RETURNED ALICE'S NOTE.

The injected condition changed the query logic and broadened the results.

comp6841-web-security-project > vulnerable_version > app.py > search

```python
@app.route("/search")
@login_required
def search():
    """Search notes owned by the current user."""
    query = request.args.get("q", "").strip()
    results = []
    sql_error = None

    if query:
        # VULNERABLE: SQL injection demonstration for this local COMP6841 app.
        # User input is directly inserted into the SQL string instead of using
        # parameterized placeholders. The fixed version will replace this.
        unsafe_sql = (
            "SELECT id, title, body, created_at "
            "FROM notes "
            f"WHERE user_id = {session['user_id']} "
            f"AND (title LIKE '%{query}%' OR body LIKE '%{query}%') "
            "ORDER BY created_at DESC"
        )

        try:
            results = get_db().execute(unsafe_sql).fetchall()
        except sqlite3.Error as error:
            sql_error = str(error)

    return render_template(
        "search.html",
```

Cause

SEARCH INPUT WAS INTERPOLATED INTO THE SQL STRING.

comp6841-web-security-project > fixed_version > app.py > ...

```python
@app.route("/search")
@login_required
def search():
    """Search notes owned by the current user."""
    query = request.args.get("q", "").strip()
    results = []

    if query:
        # FIXED: SQL injection. The vulnerable version built SQL with f-strings and
        # inserted search text directly into the query. Here we use ? placeholders
        # so the search text is sent as data, not as part of the SQL command.
        results = get_db().execute(
            """
            SELECT id, title, body, created_at
            FROM notes
            WHERE user_id = ?
              AND (title LIKE ? OR body LIKE ?)
            ORDER BY created_at DESC
            """,
            (session["user_id"], f"%{query}%", f"%{query}%"),
        ).fetchall()

    return render_template(
        "search.html",
        query=query,
        results=results,
    )
```

Fix:

PARAMETERIZED QUERY WITH ? PLACEHOLDERS.

STORED CONTENT BECAME BROWSER CODE

comp6841-web-security-project > vulnerable_version > templates > note_detail.html > ...

```
1  {% extends "base.html" %}
2
3  {% block title %}{{ note["title"] }} - VulnerableNotes{% endblock %}
4
5  {% block content %}
6    <article class="card">
7      <p><a href="{{ url_for('notes') }}">Back to my notes</a></p>
8
9      <h1>{{ note["title"] }}</h1>
10     <p class="note-meta">Created {{ note["created_at"] }}</p>
11     <p class="warning-note">
12       VULNERABLE: Note body is rendered unsafely on this page for the local
13       COMP6841 stored XSS demonstration.
14     </p>
15     <div class="note-body">
16       {# VULNERABLE: Note body is rendered with |safe so stored HTML/JS runs in the browser. #}
17       <p>{{ note["body"] | safe }}</p>
18     </div>
19   </article>
20 {% endblock %}
```

FIX:

Remove | safe and use normal Jinja escaping

Stored

NOTE BODY SAVED TO SQLITE

Rendered

JINJA | SAFE DISABLED ESCAPING

VulnerableNotes — My notes | Search | Signed in as bob | Log out

My notes

Create a note with a simple title and body.

Title

suprise mf!

Body

<script>alert('Stored XSS')</script>

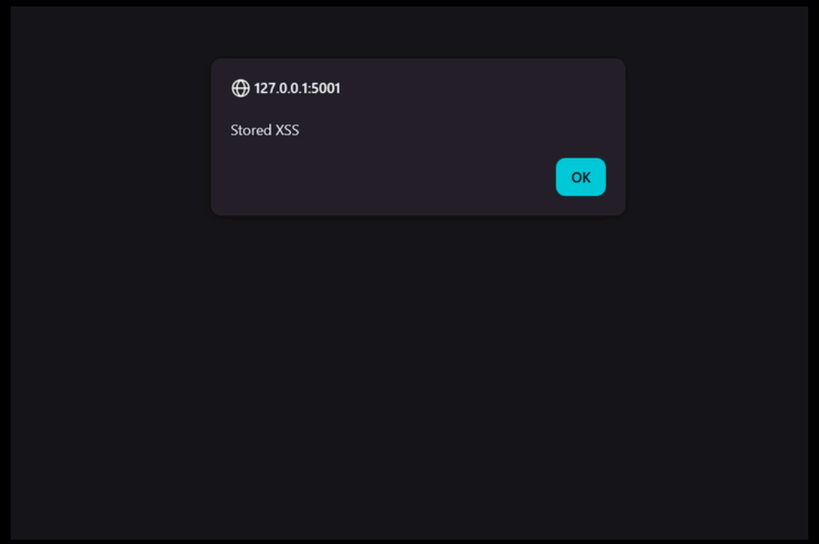
Create note

Saved notes

suprise mf!

<script>alert('Stored XSS')</script>

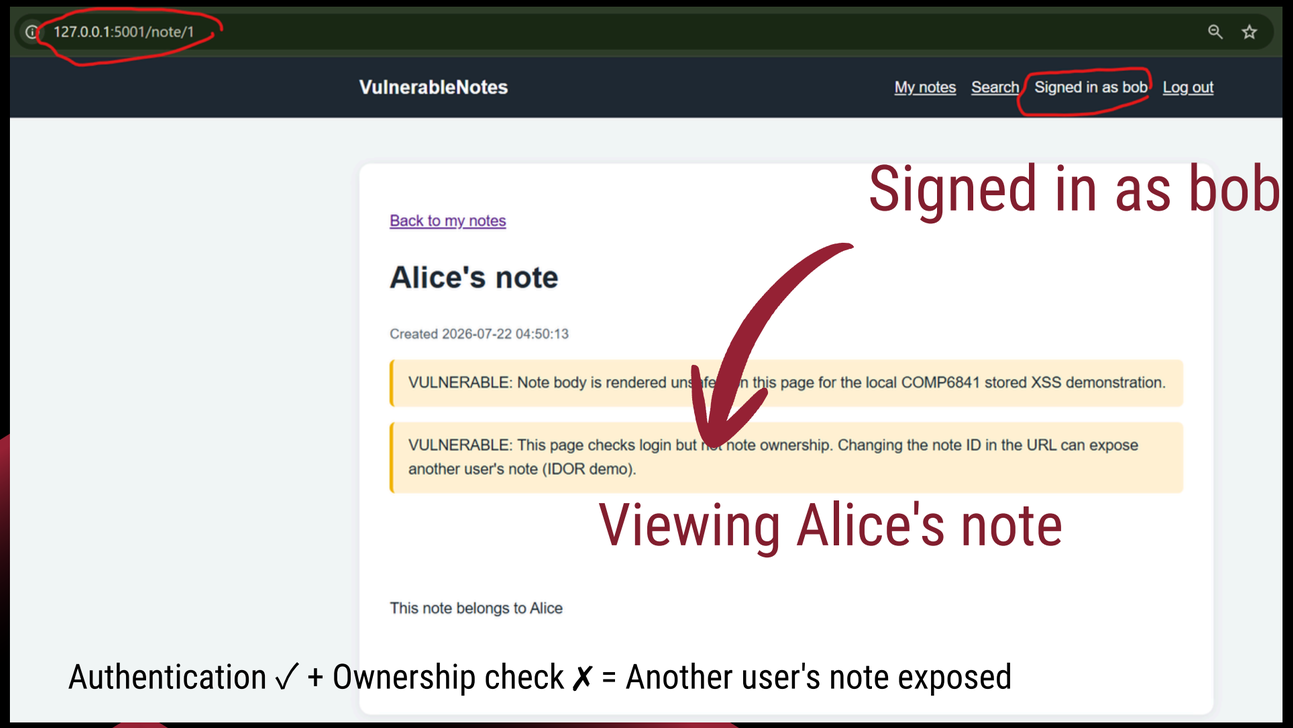
Created 2026-07-22 04:24:07



Executed

BROWSER RAN THE STORED SCRIPT

LOGIN SUCCEEDED, BUT AUTHORIZATION FAILED



Signed in as bob

Viewing Alice's note

Authenticated as Bob

Authentication ✓ + Ownership check ✗ = Another user's note exposed

```python
@app.route("/note/<int:note_id>")
@login_required
def note_detail(note_id):
    """Show one note by ID for any logged-in user."""
    # VULNERABLE: Broken access control / IDOR demonstration for this local
    # COMP6841 app. The route checks that the visitor is logged in, but does
    # not check that the note belongs to the current user. Changing the note
    # ID in the URL can expose another user's note.
    note = get_db().execute(
        """
        SELECT id, title, body, created_at
        FROM notes
        WHERE id = ?
        """,
        (note_id,),
    ).fetchone()

    if note is None:
        abort(404)

    return render_template("note_detail.html", note=note)
```

FIX: query by both note_id and the logged-in user_id

```python
@app.route("/note/<int:note_id>")
@login_required
def note_detail(note_id):
    """Show one note owned by the current user."""
    # FIXED: Broken access control / IDOR. The vulnerable version looked up notes
    # by id only. Here we also require user_id to match the logged-in user.
    note = get_db().execute(
        """
        SELECT id, title, body, created_at
        FROM notes
        WHERE id = ? AND user_id = ?
        """,
        (note_id, session["user_id"]),
    ).fetchone()

    if note is None:
        abort(404)

    return render_template("note_detail.html", note=note)
```

A DATABASE READ REVEALED THE PASSWORD DIRECTLY

VULNERABLE:
Stored and compared as plaintext

Example database value: testpwd

comp6841-web-security-project > vulnerable_version > app.py > register

```
76   def register():
88           if error is None:
89               try:
90                   db = get_db()
91                   # VULNERABLE: Passwords are stored in plaintext for the later
92                   # COMP6841 weak password handling demonstration.
93                   db.execute(
94                       "INSERT INTO users (username, password) VALUES (?, ?)",
95                       (username, password),
96                   )
97                   db.commit()
```

comp6841-web-security-project > vulnerable_version > app.py > register

```
110  def login():
121
122          if user is None:
123              error = "Incorrect username."
124          # VULNERABLE: Plaintext password comparison is intentionally kept for
125          # the later COMP6841 weak password handling demonstration.
126          elif user["password"] != password:
127              error = "Incorre      word."
128
```

A copied database would immediately reveal reusable credentials.

```
zach@LEGION5PRO:~/comp6841/comp6841-web-security-project$ python3 - <<'PY'
import sqlite3
conn = sqlite3.connect("vulnerable_version/vulnerablenotes.sqlite3")
for row in conn.execute("SELECT id, username, password FROM users"):
    print(row)
conn.close()
PY
(1, 'testacc', 'testpwd')
```

FIXED:

comp6841-web-security-project > fixed_version > app.py > note_detail

```
83   def register():
95           if error is None:
96               try:
97                   db = get_db()
98                   # FIXED: Weak password handling. The vulnerable version stored the
99                   # password string directly in SQLite. Here we store a hash instead.
100                  db.execute(
101                      "INSERT INTO users (username, password) VALUES (?, ?)",
102                      (username, generate_password_hash(password)),
103                  )
104                  db.commit()
```

Store a one-way scrypt hash

comp6841-web-security-project > fixed_version > app.py > note_detail

```
117  def login():
128
129          if user is None:
130              error = "Incorrect username."
131          # FIXED: Weak password handling. The vulnerable version compared plaintext
132          # strings with user["password"] != password. Here we verify against the hash.
133          elif not check_password_hash(user["password"], password):
134              error = "Incorrect password."
135
```

Verify with check_password_hash()

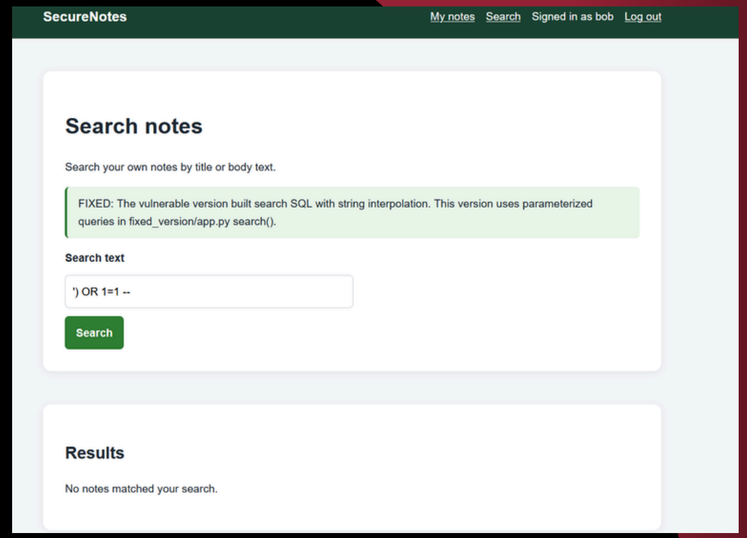
User passwords securely stored as hash

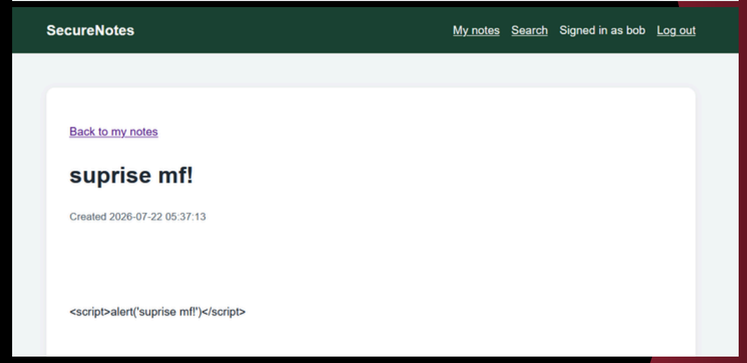
```
(.venv) zach@LEGION5PRO:~/comp6841/comp6841-web-security-project$ python3 - <<'PY'
import sqlite3
conn = sqlite3.connect("fixed_version/securenotes.sqlite3")
for row in conn.execute("SELECT id, username, password FROM users"):
    print(row)
conn.close()
PY
(1, 'alice', 'scrypt:32768:8:1$ev2ligRigHDPpd2S$e4b172095ba5016b5356213767c4a44a102754103ef2a809e815df73d888ca7c5bd9d706cef3546a1cc9d88a3612a21c33211b6e9c6de5cd72459e9a91f748f1')
(2, 'bob', 'scrypt:32768:8:1$iwL4vRhYyv9ksY8x$a6c0bd17d0e38bb8955c3aa23efcfaa820dc7f9421ee59c7869da2dfd0ee35892198d9d9e84a1ec0aafe5a55c545e6765943fdcb39cd8b3d42089347339b2952')
(.venv) zach@LEGION5PRO:~/comp6841/comp6841-web-security-project$
```

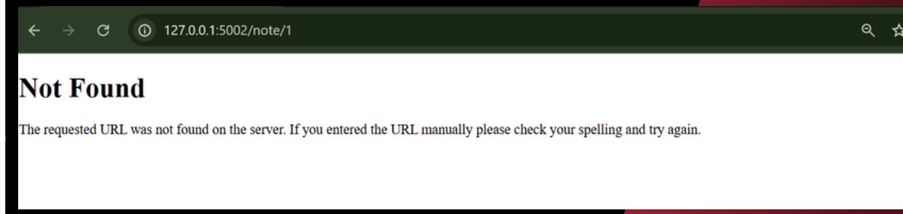
SECURENOTES BLOCKED ALL FOUR ORIGINAL DEMONSTRATIONS

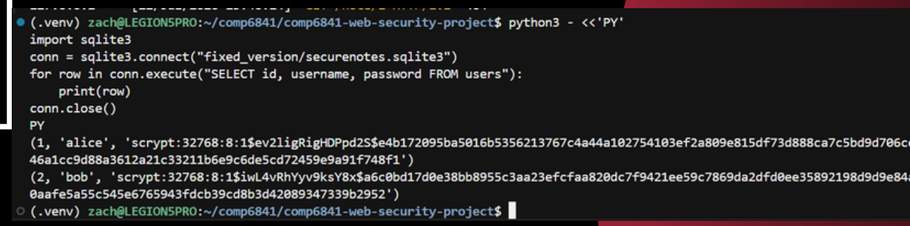
Test	Targeted change	Retest result
SQL injection	Parameterized SQL	Input treated as text; no unexpected notes
Stored XSS	Normal Jinja escaping	Payload displayed as text; no alert
IDOR	Server-side ownership check	Bob received 404 for Alice's note
Passwords	Werkzeug scrypt hashing	No readable passwords in SQLite

4/4 MANUAL RETESTS PASSED









FROM WORKING FEATURES TO SECURE DESIGN

What I achieved

- ✓ Built two working Flask/ SQLite applications
- ✓ Demonstrated and fixed four vulnerabilities
- ✓ Recorded before-and-after evidence for every test

What I learned

- Security failures can come from one ordinary coding decision.
- Authentication and authorization solve different problems.
- Controlled local tests can demonstrate impact ethically.

Treat input as data, enforce ownership on the server, and never store passwords in plaintext.

VULNERABLENOTES

THANK YOU & QUESTIONS?

https://github.com/StickyGrenade/comp6841-web-security-projects

Project: VulnerableNotes — Building, Breaking, and Securing a Local Flask Notes App |
Contact: z5602186@unsw.edu.au | Course: comp6841 security engineering